SOFTWARE ENGINEERING • UNIT 2

Complete Study Guide

Chapters 3, 5, 7 & 8

Agile Development | System Modeling | Design & Implementation | Software Testing

Overview

This guide consolidates all lecture material from Unit 2 of the Software Engineering course, covering four major chapters that build upon each other.

Chapter	Topic
Chapter 3	Agile Software Development — rapid methods, XP practices, Scrum framework
Chapter 5	System Modeling — UML diagrams, context, interaction, structural & behavioral models
Chapter 7	Design and Implementation — OO design process, architectural design, UML design models
Chapter 8	Software Testing — development testing, TDD, release testing, user testing

Chapter 3 Agile Software Development

3.1 Rapid Software Development

Modern businesses operate in fast-changing environments where stable requirements are rarely achievable. Rapid development and delivery has become the most critical requirement for many software systems. In rapid development, specification, design, and implementation are inter-leaved, and the system is built as a series of versions with stakeholder feedback at each stage.

3.2 Agile Methods

Agile methods emerged from dissatisfaction with heavyweight design methods of the 1980s and 1990s. They focus on code rather than documentation, use iterative development, and aim to deliver working software quickly while evolving to meet changing requirements.

Agile Manifesto	Individuals and interactions over processes and tools | Working software over comprehensive documentation | Customer collaboration over contract negotiation | Responding to change over following a plan

Principles of Agile Methods

Principle	Description
Customer Involvement	Customers closely involved throughout — provide and prioritize requirements, evaluate iterations.
Incremental Delivery	Software developed in increments; customer specifies what each increment includes.
People not Process	Team skills recognized and exploited; developers find their own ways of working.
Embrace Change	Design the system to accommodate changing requirements.
Maintain Simplicity	Focus on simplicity in both the software and the development process.

Agile Method Applicability

- Small or medium-sized product development for sale.
- Custom system development with committed customer involvement.
- NOT ideal for very large systems requiring larger, distributed development teams.

Problems with Agile Methods

- Difficulty keeping customer interest throughout the entire process.
- Team members may be unsuited to intense involvement required.
- Prioritizing changes is difficult when there are multiple stakeholders.
- Maintaining simplicity requires extra effort.
- Writing contracts for incremental specification is difficult.

3.3 Plan-Driven vs. Agile Development

Most projects include elements of both approaches. Plan-driven development uses separate stages with planned outputs; iteration occurs within activities. Agile inter-leaves all stages, with outputs decided through negotiation during development.

Question	Recommendation
Detailed spec needed upfront?	Use plan-driven approach.
Incremental delivery realistic?	Consider agile methods.
Small co-located team?	Agile works well.
Large distributed team?	Plan-driven is better.
External regulatory approval required?	Plan-driven (detailed documentation required).
Long-lifetime system?	Plan-driven (support team needs documentation).

3.4 Extreme Programming (XP)

Extreme Programming is the best-known agile method. New versions may be built several times per day, increments delivered to customers every 2 weeks, and all tests must pass before any build is accepted.

XP Practices

Practice	Description
Incremental Planning	Requirements on story cards; stories prioritized by time and value; broken into development tasks.

Practice	Description
Small Releases	Minimal useful functionality delivered first; frequent releases add features incrementally.
Simple Design	Only enough design for current requirements — no more.
Test-First Development	Automated tests written before implementing any functionality.
Refactoring	Continuous code improvement for clarity and maintainability, even without immediate need.
Pair Programming	Two developers work together, checking each other's work continuously.
Collective Ownership	All developers can change any code; no islands of expertise; collective responsibility.
Continuous Integration	Tasks integrated into the whole system immediately on completion; all unit tests must pass.
Sustainable Pace	No excessive overtime — it reduces code quality and medium-term productivity.
On-site Customer	Customer representative available full-time as a member of the XP team.

User Stories and Tasks

Requirements are expressed as user stories written on cards. The development team breaks stories into implementation tasks that form the basis of schedule and cost estimates. The customer chooses which stories go into the next release based on priorities and schedule estimates.

Refactoring

XP maintains that designing for anticipated future change is not worthwhile because changes cannot be reliably anticipated. Instead, constant refactoring makes changes easier when they actually occur. Examples include: re-organizing class hierarchies to remove duplicate code, renaming attributes and methods for clarity, and replacing inline code with library method calls.

Testing in XP

- Test-first development: tests written before code clarify requirements.
- Tests are executable programs (not data) that run automatically.
- All previous and new tests run automatically with each new increment.
- Customer involvement: customer helps write acceptance tests during development.
- Automated test frameworks (JUnit, Selenium) support the entire process.

XP Testing Difficulties

- Programmers prefer coding to testing and may write incomplete tests.
- Complex user interfaces make incremental unit testing difficult.
- Hard to judge completeness of a test set — coverage may be incomplete.

Pair Programming Benefits

- Develops collective ownership and spreads knowledge across the team.
- Acts as an informal review — every line of code seen by at least two people.
- Encourages refactoring since the whole team benefits immediately.
- Reduces project risk when team members leave — knowledge is shared.
- Evidence shows pair productivity is comparable to two working independently.

3.5 Agile Project Management — Scrum

Scrum is a general agile method focused on managing iterative development rather than specific agile practices. It has three phases: initial outline planning (objectives and architecture), a series of sprint cycles (each developing a system increment), and project closure (documentation and lessons learned).

Scrum Roles

Role	Responsibility
Product Owner	Defines product vision, manages and prioritizes the product backlog.
Scrum Master	Facilitates the Scrum process, removes blockers, shields the team from external distractions.
Development Team	Cross-functional team (developers, designers, testers) that builds the product.

The Sprint Cycle

- Sprints are fixed-length (2–4 weeks), each producing a working system increment.
- Planning input is the product backlog — the full prioritized list of work.
- Selection phase: team and customer choose features for the sprint together.
- During the sprint, the team is isolated; all communications go through the Scrum Master.
- At sprint end, work is reviewed and presented to stakeholders; next sprint then begins.

Daily Scrum Meeting

- Short daily meeting where every team member shares information.
- Each member answers: What did I do yesterday? What will I do today? Any blockers?
- Keeps everyone informed and enables rapid re-planning if problems arise.

Scrum Benefits

- Product broken into manageable, understandable chunks.
- Unstable requirements do not block progress.

- Full team visibility improves communication.
- Customers see on-time delivery of increments and provide rapid feedback.
- Trust built between customers and developers; positive culture created.

Chapter 5 System Modeling

5.1 What is System Modeling?

System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective. It now means representing a system using graphical notation — almost always based on UML — to help analysts understand system functionality and communicate with customers.

Use of Models	Notes
Facilitate Discussion	Incomplete or incorrect models are acceptable — their role is to support discussion.
Document Existing Systems	Models should accurately represent the system, but need not be complete.
Generate Implementation	Models must be both correct and complete.

System Perspectives

- External perspective: context or environment of the system (physical, regulatory, technological, stakeholders).
- Interaction perspective: interactions between system and environment, or between system components.
- Structural perspective: organization of the system or structure of the data it processes.
- Behavioral perspective: dynamic behavior and how the system responds to events.

UML Diagram Types

Diagram Type	Purpose & Category
Activity Diagrams	Activities involved in a process or data processing. Used in Context Models.
Use Case Diagrams	Interactions between a system and its environment. Used in Interaction Models.
Sequence Diagrams	Interactions between actors, system, and components over time. Used in Interaction Models.
Class Diagrams	Object classes and the associations between them. Used in Structural Models.
State Diagrams	How the system reacts to internal and external events. Used in Behavioral Models.

5.2 Context Models

At early specification stages, the system boundary must be decided in collaboration with stakeholders. The boundary position has a profound effect on requirements and is partly a political judgment. Context models show other systems in the environment. Process models (UML activity diagrams) reveal how the system is used in broader business processes.

5.3 Interaction Models

Interaction modeling is important for identifying user requirements, highlighting system-to-system communication problems, and understanding if a proposed structure will deliver required performance and dependability. Use case diagrams and sequence diagrams are the primary tools.

Use Case Modeling

- Each use case represents a discrete task involving external interaction with a system.
- Actors may be people or other systems.
- Shown diagrammatically for an overview and in detailed textual form for specifics.

Sequence Diagrams

Sequence diagrams model interactions between actors and objects during a particular use case. Objects are listed along the top with vertical dotted lifelines; interactions are shown as annotated arrows read top-to-bottom.

5.4 Structural Models

Structural models display the organization of a system in terms of its components and relationships. Static models show components, attributes, and relationships. Dynamic models show interactions and communications during runtime.

Class Diagrams

- Used in OO system modeling to show classes and associations.
- An association is a link between classes indicating a relationship.
- Multiplicity indicators (1, 1..*, 0..*) show how many objects can be in each relationship.
- Generalization (inheritance) allows lower-level classes to inherit attributes and operations from higher-level classes.
- Aggregation models show how collection classes are composed of other classes (part-of relationship).

5.5 Behavioral Models

Behavioral models show what happens when a system responds to stimuli from its environment. Stimuli are of two types: data (data arrives to be processed) and events (something happens that triggers processing).

Data-Driven Modeling

Data-driven models show the sequence of actions involved in processing input data and generating output. Useful during requirements analysis to show end-to-end processing. Represented using UML activity diagrams.

Event-Driven Modeling — State Machine Models

Real-time systems are often event-driven with minimal data processing. State machine models show system response to events. The system has a finite number of states; events cause transitions between them. Statecharts in UML represent state machine models.

- States shown as rounded rectangles (nodes).
- Events shown as labeled arrows (arcs) between nodes.
- Initial state shown as a filled black circle.

Chapter 7 Design and Implementation

7.1 Overview

Software design and implementation is the stage where an executable software system is developed. Design and implementation activities are inter-leaved. Design is the creative activity of identifying software components and their relationships based on requirements. Implementation is the process of realizing the design as a program.

Build or Buy?	Off-the-shelf (COTS) systems can often be adapted to user requirements. When using COTS, the design process focuses on configuring the system rather than building from scratch — cheaper and faster for many domains.

7.2 Object-Oriented Design Process

OO design involves designing object classes and the relationships between them. Objects are created dynamically at runtime from class definitions. For large systems developed by different groups, design models are critical communication mechanisms.

Process Stages

- Understand and define the context and external interactions with the system.
- Design the system architecture.
- Identify the principal system objects.
- Develop design models.
- Specify object interfaces.

7.3 System Context and Interactions

Understanding the relationships between the software being designed and its external environment is essential. It helps determine how to provide required functionality and how to structure the system. It also helps establish system boundaries.

Model Type	Description
System Context Model	Structural model showing other systems in the environment of the system being developed.
Interaction Model	Dynamic model showing how the system interacts with its environment as it is used.

7.4 Architectural Design

Once system interactions are understood, this information is used to design the system architecture. Major components and their interactions are identified, then organized using an architectural pattern (e.g., layered, client-server).

Weather Station example: six independent subsystems (Fault Manager, Configuration Manager, Power Manager, Communications Subsystem, Data Collection Subsystem, Instruments Subsystem) all connected via a central Communication Link.

7.5 Object Class Identification

Identifying object classes is the most difficult part of OO design. There is no formula — it relies on skill, experience, and domain knowledge. Object identification is iterative.

Approaches to Identification

- Grammatical analysis of a natural language description: objects/attributes are nouns; operations are verbs.
- Tangible entities: things, roles, events, interactions, locations, organizational units in the application domain.
- Scenario-based analysis: analyze scenarios of system use to identify required objects, attributes, and operations.

7.6 Design Models

Design models show objects, classes, and their relationships. Static models (class diagrams, object diagrams, component diagrams, deployment designs) describe structure. Dynamic models (sequence diagrams, activity diagrams, state diagrams) describe interactions.

Model Type	Description
Subsystem Models	Logical groupings of objects into coherent subsystems. Example: Shopping subsystem, User Management subsystem.

Model Type	Description
Sequence Models	Show the sequence of object interactions. Objects arranged horizontally; time flows vertically; interactions are labeled arrows.
State Machine Models	Show how individual objects change state in response to events.

7.7 Interface Specification

Object interfaces should be defined precisely so different parts of the system can be developed simultaneously. An interface should not include data storage details but should define methods to access and update data. An object can have multiple interfaces. UML represents interfaces with the <<interface>> stereotype on class diagrams.

Chapter 8 Software Testing

8.1 Program Testing Fundamentals

Testing shows that a program does what it is intended to do and discovers defects before use. It executes a program with artificial data and checks results for errors, anomalies, and non-functional behavior. Testing can reveal the presence of errors — it cannot prove their absence.

Goal	Description
Validation Testing	Demonstrate the software meets requirements. Tests reflect expected use. A successful test shows the system works as intended.
Defect Testing	Discover faults where behavior is incorrect or non-conformant. Test cases can be deliberately obscure. A successful test exposes a defect.

V&V	Verification: 'Are we building the product right?' — software conforms to its specification. Validation: 'Are we building the right product?' — software does what the user really requires.

8.2 Inspections vs. Testing

Software inspections analyze the static system representation to discover problems (static verification) without executing the system. Software testing exercises the product's behavior (dynamic verification) by running the system with test data. Both should be used throughout the V&V process.

Advantages of Inspections

- Errors cannot mask other errors — no interaction between errors in a static process.
- Incomplete system versions can be inspected at no additional cost.
- Can check quality attributes: standards compliance, portability, maintainability.
- Can identify inefficiencies, inappropriate algorithms, poor programming style.

Limitations: Inspections cannot check conformance with the customer's real requirements or check non-functional characteristics such as performance and usability.

8.3 Software Testing Process

The testing process cycle: Design Test Cases → Prepare Test Data → Run Program with Test Data → Compare Results to Test Cases → Generate Test Reports. If results are incorrect, the cycle repeats after fixing issues.

8.4 Stages of Testing

Stage	Description
Development Testing	Carried out by the development team during development to discover bugs and defects.
Release Testing	Separate testing team tests a complete version before release to users.
User Testing	Users test the system in their own environment to validate real-world suitability.

8.5 Development Testing

Unit Testing

Unit testing tests individual program units or object classes in isolation. It is a defect testing process. Units may be individual functions/methods, object classes, or composite components with defined interfaces.

- Test all operations associated with an object.
- Set and interrogate all object attributes.
- Exercise the object in all possible states.
- Inheritance makes testing harder as information is not localized.

Automated Testing

Unit testing should be automated using a test automation framework (JUnit, Selenium). Automated test components have three parts:

- Setup: initialize the system with the test inputs and expected outputs.
- Call: call the object or method to be tested.
- Assertion: compare the result with the expected result — True = success, False = failure.

Testing Strategies

Strategy	Description
Partition Testing	Identify equivalence partitions — groups of inputs processed the same way. Test one value from each partition. Reduces test cases while ensuring good coverage.

Strategy	Description
Guideline-based Testing	Use guidelines based on experience of common programmer mistakes (boundary cases, buffer overflow, repeated inputs, invalid outputs, extreme computation results).

Equivalence Partitioning Example: A program accepting 4–8 inputs of five-digit integers > 10,000. Partitions by number of inputs: < 4, 4–8, > 8. Partitions by value: < 10,000, exactly 10,000, five-digit integer > 10,000.

Component Testing

Tests composite components made up of several interacting objects. Focus is on showing that component interfaces behave according to specification. Interface types: parameter interfaces, shared memory interfaces, procedural interfaces, message passing interfaces.

Interface Errors

- Interface misuse: calling component makes an error in using the interface (e.g., wrong parameter order).
- Interface misunderstanding: calling component has incorrect assumptions about called component's behavior.
- Timing errors: components operating at different speeds causing access to out-of-date information.

System Testing

System testing integrates components and tests the integrated system, focusing on interactions between components: compatibility, correct interaction, and correct data transfer. Tests the emergent behavior of a system. Use-case based testing uses the system's use cases to drive system test design.

8.6 Test-Driven Development (TDD)

TDD inter-leaves testing and code development. Tests are written before code, and passing the tests is the critical driver of development. Code is developed incrementally — never move to the next increment until the current code passes its test.

TDD Process Activities

- Identify the small increment of functionality required.
- Write an automated test for this functionality.
- Run the test — it will fail (code not yet implemented).
- Implement the functionality and re-run the test.
- Once all tests pass, move on to the next increment.

Benefits of TDD

Benefit	Explanation
Code Coverage	Every code segment has at least one associated test case — no untested code.
Regression Testing	A regression test suite is developed incrementally alongside the program.
Simplified Debugging	When a test fails, the problem is clearly in the newly written code.
System Documentation	Tests describe what the code should do — they serve as living documentation.

Regression Testing

Regression testing checks that changes have not broken previously working code. With automated testing, all tests rerun every time a change is made. Tests must run successfully before any change is committed.

8.7 Release Testing

Release testing tests a particular release intended for external use. The goal is to convince the supplier that the system is good enough for use — demonstrating specified functionality, performance, dependability, and normal-use reliability. Usually black-box testing (tests derived only from the system specification).

Type	Focus & Team
System Testing (Dev Team)	Focuses on discovering bugs. Done by developers during development.
Release Testing	Validates the system meets requirements and is good for external use. Done by a separate team.

Requirements-Based Testing

Each requirement is examined and one or more tests are developed for it. Example — allergy warning requirement: test with no allergies (confirm no warning issued), test with known allergy (confirm warning issued), test with multiple allergies (confirm correct warnings), test overriding warning (confirm reason required).

Scenario Testing

Typical usage scenarios are invented and used to derive test cases. A scenario describes how a user interacts with the system in a realistic situation. It naturally tests multiple features and interactions simultaneously (e.g., authentication, record retrieval, drug database lookup, encryption/decryption).

Performance Testing

Tests emergent system properties such as performance and reliability. Tests should reflect actual usage profiles. Performance tests steadily increase load until performance becomes unacceptable. Stress testing deliberately overloads the system to test failure behavior and fault tolerance.

8.8 User Testing

User testing is a stage where users or customers provide input and advice. It is essential even after comprehensive system and release testing because the user's working environment affects reliability, performance, usability, and robustness in ways that cannot be replicated in a test environment.

Type	Description
Alpha Testing	Users work with the development team at the developer's site.
Beta Testing	A release is made available externally for users to experiment with and report bugs before official launch.
Acceptance Testing	Customers test the system to decide if it is ready to be accepted and deployed in their environment.

Acceptance Testing Process

- Define acceptance criteria.
- Plan acceptance testing (schedule, resources, strategy).
- Derive acceptance tests from the criteria.
- Run acceptance tests.
- Negotiate test results with stakeholders.
- Accept or reject the system.

8.9 Agile Methods and Acceptance Testing

In agile methods, the user/customer is part of the development team and is responsible for acceptance decisions. Tests are defined by the user and integrated with other automated tests — run automatically when changes are made. There is no separate acceptance testing process. A key concern is whether the embedded user truly represents all system stakeholders.

Quick Reference Key Concepts & Terms

Term	Definition
COTS	Commercial Off-The-Shelf — ready-made software/hardware available for purchase and adaptation instead of custom development.
Refactoring	Restructuring existing code without changing external behavior, to improve readability and maintainability.
Sprint	A fixed-length iteration (2–4 weeks) in Scrum that produces a working system increment.
Product Backlog	The full prioritized list of work to be done on a Scrum project; input to sprint planning.
TDD	Test-Driven Development — write tests before code; tests are the critical driver of development.
Regression Testing	Re-running all tests after changes to ensure previously working code is not broken.
Equivalence Partition	A group of inputs all processed the same way; choose one test value per partition.
Use Case	A discrete task involving external interaction with a system; actors may be people or systems.
UML	Unified Modeling Language — graphical notation standard for system modeling.
V&V	Verification and Validation — building the product right, and building the right product.
Black-box Testing	Testing based only on the specification, without knowledge of internal implementation.
Pair Programming	Two programmers work together at one workstation, reviewing each other's work continuously.
Static Verification	Inspections — checking documents and code without executing the program.
Dynamic Verification	Testing — checking behavior by executing the program with test data.
Stress Testing	Deliberately overloading the system beyond its design limits to test failure behavior.